

Module 3 - Database Integration - PostgreSQL, MySQL & MongoDB

Given data - Common Scenario: Student Course Registration System

HANDS-ON 2 - Writing SQL Queries — DML, Joins & Aggregations

Task 1: Insert, Update and Delete Data

Step 16: Insert 2 new students

SQL:

INSERT INTO students

(first_name, last_name, email, date_of_birth,
department_id, enrollment_year)

VALUES

('Nandhini', 'Subramaniam', 'nandhini@college.edu',
'2004-06-15', 1, 2023),

('Rahul', 'Kumar', 'rahul.kumar@college.edu', '2003-12-20',
2, 2022);

EXAMPLE:

< Input

Run SQL

```
SELECT* FROM students
```

Output

student_id	first_name	last_name	email	date_of_birth	department_id	enrollment_year
1	Arjun	Mehta	arjun.mehta@college.edu	2003-04-12	1	2022
2	Priya	Suresh	priya.suresh@college.edu	2003-07-25	1	2022
3	Rohan	Verma	rohan.verma@college.edu	2002-11-08	2	2021
4	Sneha	Patel	sneha.patel@college.edu	2004-01-30	3	2023
5	Vikram	Das	vikram.das@college.edu	2003-09-14	1	2022
6	Kavya	Menon	kavya.menon@college.edu	2002-05-17	2	2021
7	Aditya	Singh	aditya.singh@college.edu	2004-03-22	4	2023

< Input

Run SQL

```
SELECT* FROM students|
```

Output

4	Sneha	Patel	sneha.patel@college.edu	2004-01-30	3	2023
5	Vikram	Das	vikram.das@college.edu	2003-09-14	1	2022
6	Kavya	Menon	kavya.menon@college.edu	2002-05-17	2	2021
7	Aditya	Singh	aditya.singh@college.edu	2004-03-22	4	2023
8	Deepika	Rao	deepika.rao@college.edu	2003-08-09	1	2022
9	Nandhini	Subramaniam	nandhini@college.edu	2004-06-15	1	2023
10	Rahul	Kumar	rahul.kumar@college.edu	2003-12-20	2	2022

Step 17: Update grade

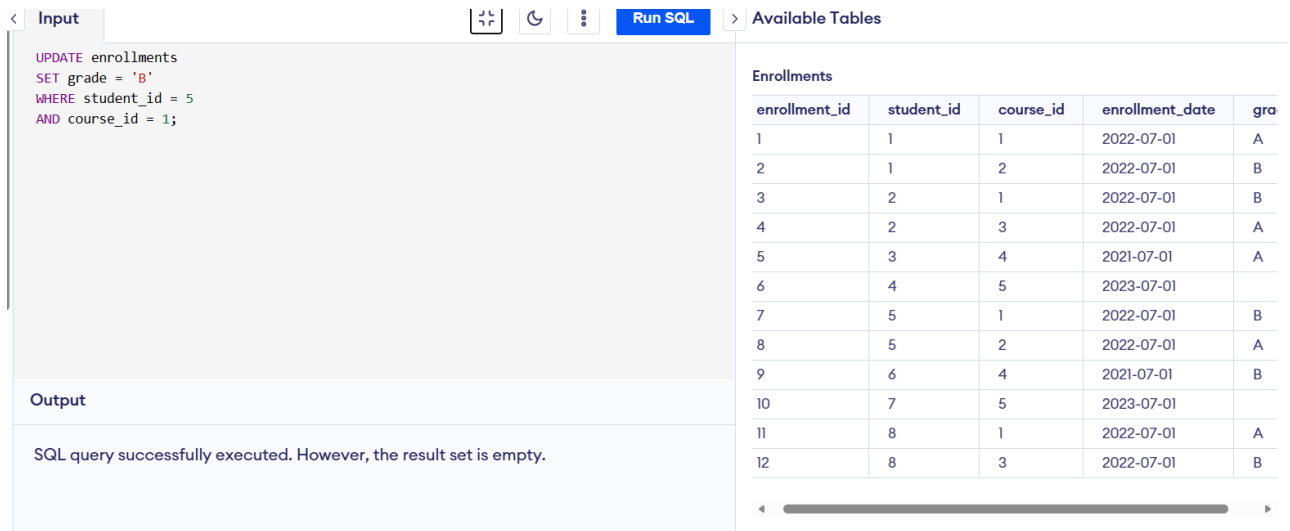
UPDATE enrollments

SET grade = 'B'

WHERE student_id = 5

AND course_id = 1;

EXAMPLE:



The screenshot shows a SQL IDE interface. On the left, the 'Input' tab contains an SQL query: `UPDATE enrollments SET grade = 'B' WHERE student_id = 5 AND course_id = 1;`. Below the input, the 'Output' tab shows the message: 'SQL query successfully executed. However, the result set is empty.' On the right, the 'Available Tables' section lists the 'Enrollments' table. Below this, a table displays the data for the 'Enrollments' table.

enrollment_id	student_id	course_id	enrollment_date	grade
1	1	1	2022-07-01	A
2	1	2	2022-07-01	B
3	2	1	2022-07-01	B
4	2	3	2022-07-01	A
5	3	4	2021-07-01	A
6	4	5	2023-07-01	
7	5	1	2022-07-01	B
8	5	2	2022-07-01	A
9	6	4	2021-07-01	B
10	7	5	2023-07-01	
11	8	1	2022-07-01	A
12	8	3	2022-07-01	B

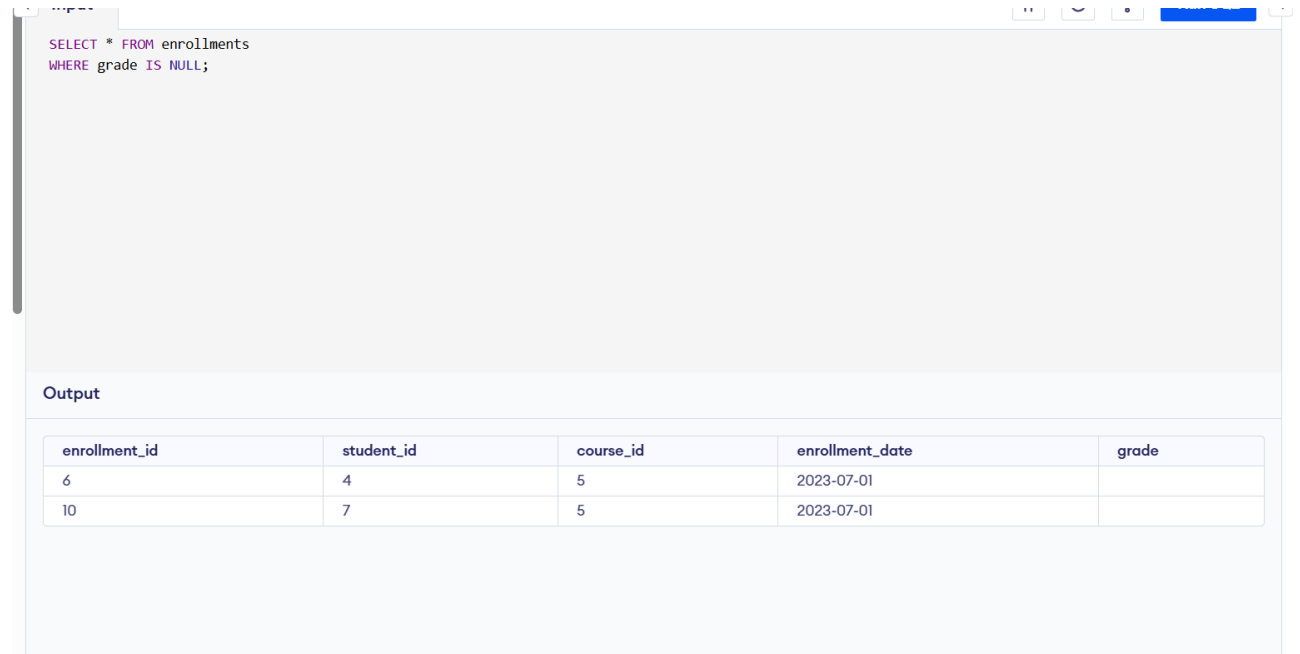
Step 18: Delete enrollments with NULL grade

First check:

SELECT * FROM enrollments

WHERE grade IS NULL;

EXAMPLE TABLE:



The screenshot shows a SQL IDE window with a query editor and an output pane. The query editor contains the following SQL statement:

```
SELECT * FROM enrollments
WHERE grade IS NULL;
```

The output pane, labeled "Output", displays the results of the query in a table format. The table has five columns: enrollment_id, student_id, course_id, enrollment_date, and grade. There are two rows of data shown.

enrollment_id	student_id	course_id	enrollment_date	grade
6	4	5	2023-07-01	
10	7	5	2023-07-01	

Then delete:

DELETE FROM enrollments

WHERE grade IS NULL;

EXAMPLE:

SELECT * FROM enrollments

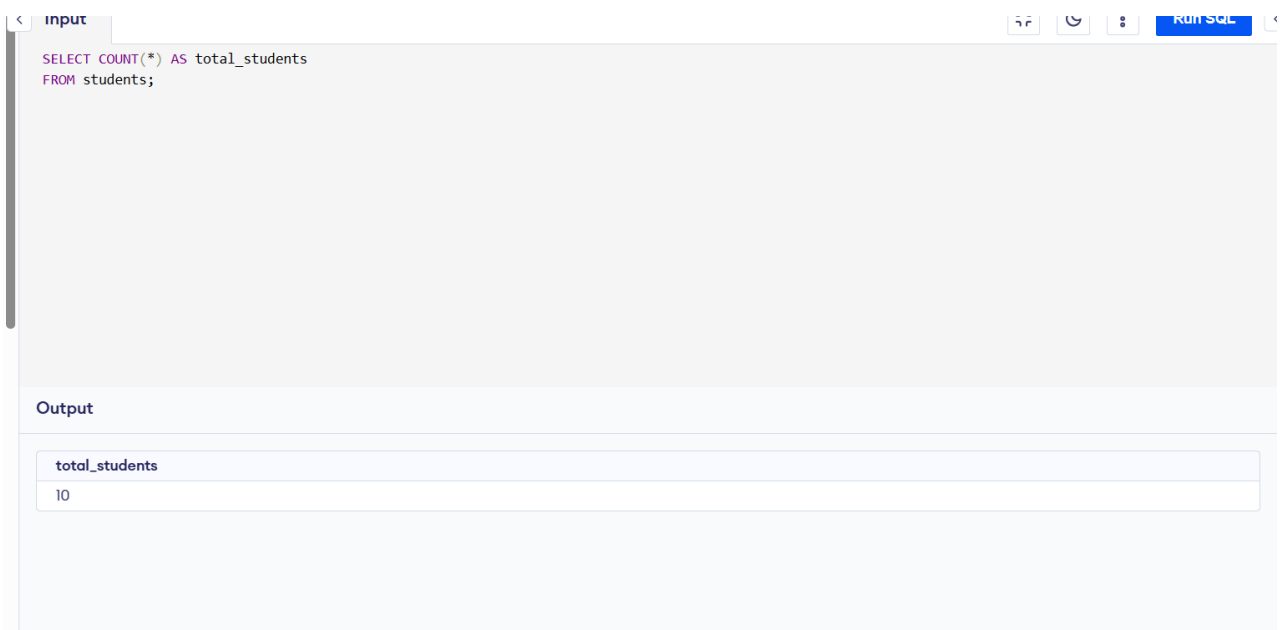
Output

enrollment_id	student_id	course_id	enrollment_date	grade
1	1	1	2022-07-01	A
2	1	2	2022-07-01	B
3	2	1	2022-07-01	B
4	2	3	2022-07-01	A
5	3	4	2021-07-01	A
7	5	1	2022-07-01	B
8	5	2	2022-07-01	A
9	6	4	2021-07-01	B

Step 19: Verify counts

SELECT COUNT(*) AS total_students
FROM students;

EXAMPLE:



The screenshot shows a SQL IDE interface. At the top, there is a tab labeled 'input'. Below the tab, the SQL query is entered: `SELECT COUNT(*) AS total_students
FROM students;`. To the right of the query editor, there are icons for undo, redo, and a 'RunSQL' button. Below the query editor, there is an 'Output' section. It contains a table with one column named 'total_students' and one row with the value '10'.

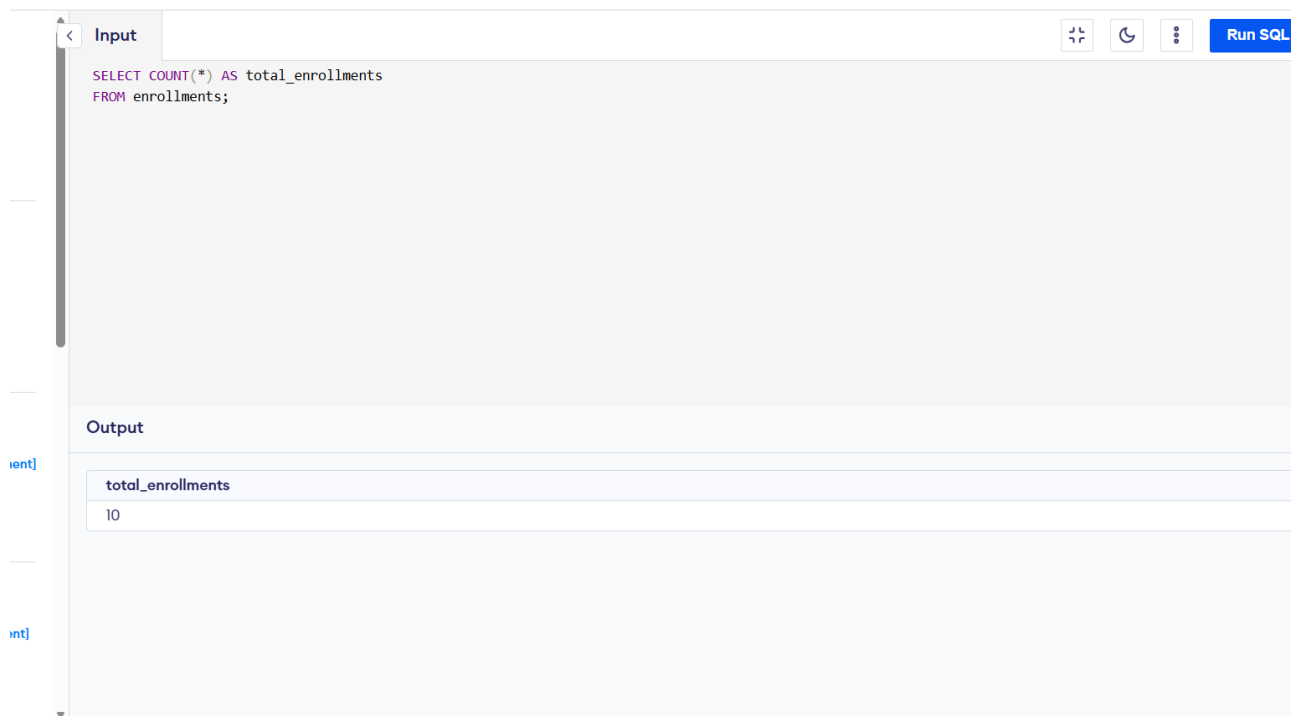
```
SELECT COUNT(*) AS total_students  
FROM students;
```

total_students
10

For enrollments:

```
SELECT COUNT(*) AS total_enrollments  
FROM enrollments;
```

EXAMPLE



The screenshot shows a SQL query editor interface. The 'Input' tab is active, displaying the following SQL query:

```
SELECT COUNT(*) AS total_enrollments
FROM enrollments;
```

Below the input area, the 'Output' tab shows the result of the query in a table format:

total_enrollments
10

- Students = **10**
- Enrollments = **10** (after deleting the 2 NULL-grade rows)

Task 2: Single-Table Queries and Filtering

Goal: Build confidence with SELECT, WHERE, ORDER BY, and LIKE

SELECT *

FROM students

WHERE enrollment_year = 2022

ORDER BY last_name ASC;

EXAMPLE

input

```
SELECT *
FROM students
WHERE enrollment_year = 2022
ORDER BY last_name ASC;
```

Output

student_id	first_name	last_name	email	date_of_birth	department_id	enrollment_year
5	Vikram	Das	vikram.das@college.edu	2003-09-14	1	2022
10	Rahul	Kumar	rahul.kumar@college.edu	2003-12-20	2	2022
1	Arjun	Mehta	arjun.mehta@college.edu	2003-04-12	1	2022
8	Deepika	Rao	deepika.rao@college.edu	2003-08-09	1	2022
2	Priya	Suresh	priya.suresh@college.edu	2003-07-25	1	2022

21. Find all courses with more than 3 credits

SELECT *

FROM courses

WHERE credits > 3

ORDER BY credits DESC;

EXAMPLE:

Input

```
SELECT *
FROM students
WHERE enrollment_year = 2022
ORDER BY last_name ASC;
```

Run SQL

Output

student_id	first_name	last_name	email	date_of_birth	department_id	enrollment_year
5	Vikram	Das	vikram.das@college.edu	2003-09-14	1	2022
10	Rahul	Kumar	rahul.kumar@college.edu	2003-12-20	2	2022
1	Arjun	Mehta	arjun.mehta@college.edu	2003-04-12	1	2022
8	Deepika	Rao	deepika.rao@college.edu	2003-08-09	1	2022
2	Priya	Suresh	priya.suresh@college.edu	2003-07-25	1	2022

22. List all professors whose salary is between 80,000 and 95,000

SQL

SELECT *

FROM professors

WHERE salary BETWEEN 80000 AND 95000;

EXAMPLE

Input

```
SELECT *
FROM professors
WHERE salary BETWEEN 80000 AND 95000;
```

Run SQL

Output

professor_id	prof_name	email	department_id	salary
	Dr. Anand Krishnan	anand.k@college.edu	1	95000
	Dr. Meena Pillai	meena.p@college.edu	1	88000
	Dr. Sunil Rajan	sunil.r@college.edu	2	82000

23. Find all students whose email ends with '@college.edu'

SQL

SELECT *

FROM students

WHERE email LIKE ' %@college.edu';

EXAMPLE TABLE

Input		Run SQL				
<pre>SELECT * FROM students WHERE email LIKE '%@college.edu';</pre>						
Output						
student_id	first_name	last_name	email	date_of_birth	department_id	enrollment_year
1	Arjun	Mehta	arjun.mehta@college.edu	2003-04-12	1	2022
2	Priya	Suresh	priya.suresh@college.edu	2003-07-25	1	2022
3	Rohan	Verma	rohan.verma@college.edu	2002-11-08	2	2021
4	Sneha	Patel	sneha.patel@college.edu	2004-01-30	3	2023
5	Vikram	Das	vikram.das@college.edu	2003-09-14	1	2022
6	Kavya	Menon	kavya.menon@college.edu	2002-05-17	2	2021
7	Aditya	Singh	aditya.singh@college.edu	2004-03-22	4	2023
8	Deepika	Rao	deepika.rao@college.edu	2003-08-09	1	2022

24. Count the total number of students per enrollment_year

SQL

```
SELECT enrollment_year,
       COUNT(*) AS total_students
FROM students
GROUP BY enrollment_year;
```

EXAMPLE TABLE

Input

Run SQL

```
SELECT enrollment_year,  
       COUNT(*) AS total_students  
FROM students  
GROUP BY enrollment_year;
```

Output

enrollment_year	total_students
2021	2
2022	5
2023	3

Task 3: Multi-Table Joins

Goal: Write JOIN queries spanning 2–4 tables to answer real business questions.

25. List each student's full name along with their department name

SQL

```
SELECT CONCAT(s.first_name, ' ', s.last_name) AS  
student_name,
```

```
       d.dept_name
```

```
FROM students s
```

```
JOIN departments d
```

```
ON s.department_id = d.department_id;
```

EXAMPLE TABLE

Input

Run SQL

```
SELECT s.first_name || ' ' || s.last_name AS student_name,
       d.dept_name
FROM students s
JOIN departments d
ON s.department_id = d.department_id;
```

Output

student_name	dept_name
Arjun Mehta	Computer Science
Priya Suresh	Computer Science
Rohan Verma	Electronics
Sneha Patel	Mechanical
Vikram Das	Computer Science
Kavya Menon	Electronics
Aditya Singh	Civil

```
SELECT s.first_name || ' ' || s.last_name AS student_name,
       d.dept_name
FROM students s
JOIN departments d
ON s.department_id = d.department_id;
```

Output

Sneha Patel	Mechanical
Vikram Das	Computer Science
Kavya Menon	Electronics
Aditya Singh	Civil
Deepika Rao	Computer Science
Nandhini Subramaniam	Computer Science
Rahul Kumar	Electronics

26. Show each enrollment with student name and course name

SQL

SELECT e.enrollment_id,

```

s.first_name || ' ' || s.last_name AS student_name,
c.course_name,
e.grade
FROM enrollments e
JOIN students s
ON e.student_id = s.student_id
JOIN courses c
ON e.course_id = c.course_id;

```

EXAMPLE TABLE

Input Run SQL			
<pre> SELECT e.enrollment_id, s.first_name ' ' s.last_name AS student_name, c.course_name, e.grade FROM enrollments e JOIN students s ON e.student_id = s.student_id JOIN courses c ON e.course_id = c.course_id; </pre>			
Output			
enrollment_id	student_name	course_name	grade
1	Arjun Mehta	Data Structures & Algorithms	A
2	Arjun Mehta	Database Management Systems	B
3	Priya Suresh	Data Structures & Algorithms	B
4	Priya Suresh	Object Oriented Programming	A
5	Rohan Verma	Circuit Theory	A
7	Vikram Das	Data Structures & Algorithms	B
8	Vikram Das	Database Management Systems	A
9	Kavya Menon	Circuit Theory	B

27. Find students who are NOT enrolled in any course

SQL

```

SELECT s.student_id,

```

```
s.first_name || ' ' || s.last_name AS student_name
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

EXAMPLE TABLE

Input

```
SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Run SQL

Output

student_id	student_name
4	Sneha Patel
7	Aditya Singh
9	Nandhini Subramaniam
10	Rahul Kumar

28. Display every course with the number of students enrolled

SQL

```
SELECT c.course_name,
       COUNT(e.student_id) AS enrollment_count
```

```
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name;
```

EXAMPLE TABLE

```
SELECT c.course_name,
       COUNT(e.student_id) AS enrollment_count
FROM   courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name;
```

Output

course_name	enrollment_count
Data Structures & Algorithms	4
Database Management Systems	2
Object Oriented Programming	2
Circuit Theory	2
Thermodynamics	0

Step 29: List each department with its professors and salaries

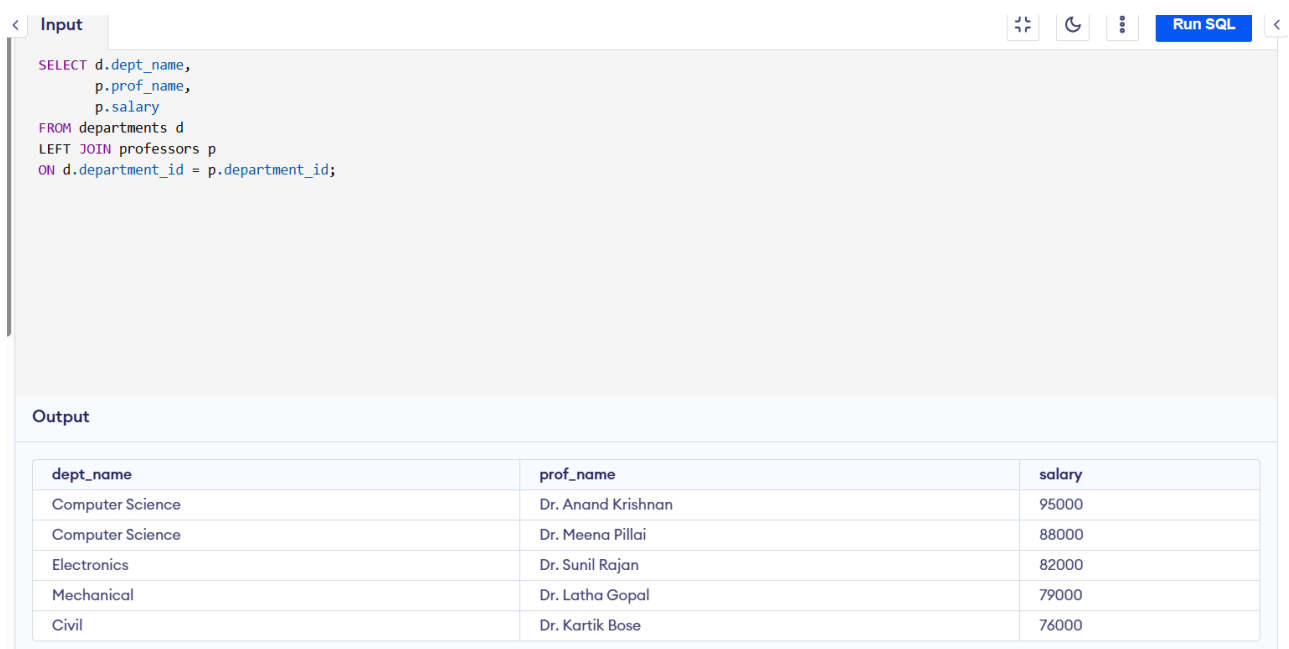
SQL

```
SELECT d.dept_name,
       p.prof_name,
```



```
p.salary  
  
FROM departments d  
  
LEFT JOIN professors p  
  
ON d.department_id = p.department_id;
```

EXAMPLE TABLE



The screenshot shows a SQL query editor interface. At the top, there's a tab labeled 'Input'. Below it, the following SQL query is entered:

```
SELECT d.dept_name,  
       p.prof_name,  
       p.salary  
FROM departments d  
LEFT JOIN professors p  
ON d.department_id = p.department_id;
```

On the right side of the editor, there are icons for undo, redo, and a 'Run SQL' button. Below the input area, there's an 'Output' section. It contains a table with the following data:

dept_name	prof_name	salary
Computer Science	Dr. Anand Krishnan	95000
Computer Science	Dr. Meena Pillai	88000
Electronics	Dr. Sunil Rajan	82000
Mechanical	Dr. Latha Gopal	79000
Civil	Dr. Kartik Bose	76000

Expected Outcome: Query 3 returns students with no enrollments. Query 4 shows all 5 courses, including those with 0 students.

Task 4: Aggregations and Grouping

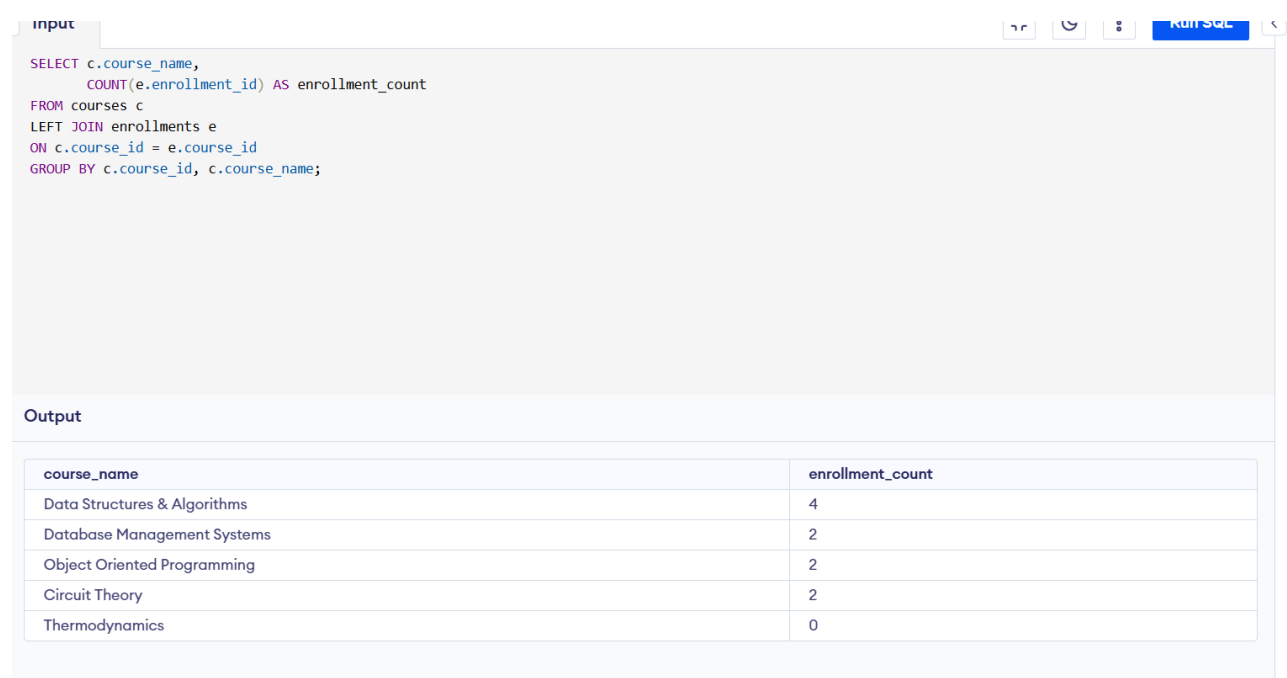
Goal: Use aggregate functions to generate summary reports. Steps:

30. Total number of enrollments per course

SQL

```
SELECT c.course_name,  
       COUNT(e.enrollment_id) AS enrollment_count  
FROM courses c  
LEFT JOIN enrollments e  
ON c.course_id = e.course_id  
GROUP BY c.course_id, c.course_name;
```

EXAMPLE TABLE



The screenshot shows a SQL query editor with a tab labeled 'input'. The query is the same as the one above. Below the query, there is an 'Output' section displaying a table with two columns: 'course_name' and 'enrollment_count'. The table contains five rows of data.

course_name	enrollment_count
Data Structures & Algorithms	4
Database Management Systems	2
Object Oriented Programming	2
Circuit Theory	2
Thermodynamics	0

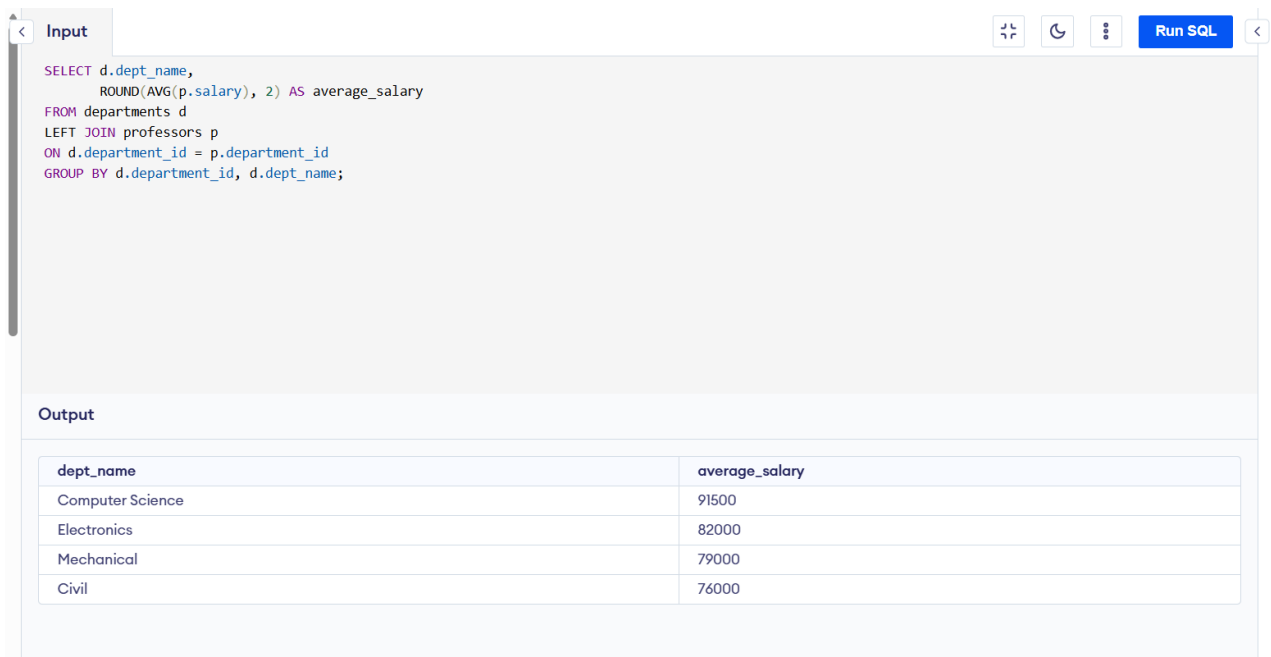
31. Average salary of professors per department

SQL

```
SELECT d.dept_name,  
       ROUND(AVG(p.salary), 2) AS average_salary
```

```
FROM departments d
LEFT JOIN professors p
ON d.department_id = p.department_id
GROUP BY d.department_id, d.dept_name;
```

EXAMPLE TABLE



The screenshot shows a SQL query editor with an 'Input' tab. The query is a SQL statement that selects department names and the average salary of professors in each department. The output is displayed in a table with two columns: 'dept_name' and 'average_salary'.

```
SELECT d.dept_name,
       ROUND(AVG(p.salary), 2) AS average_salary
FROM departments d
LEFT JOIN professors p
ON d.department_id = p.department_id
GROUP BY d.department_id, d.dept_name;
```

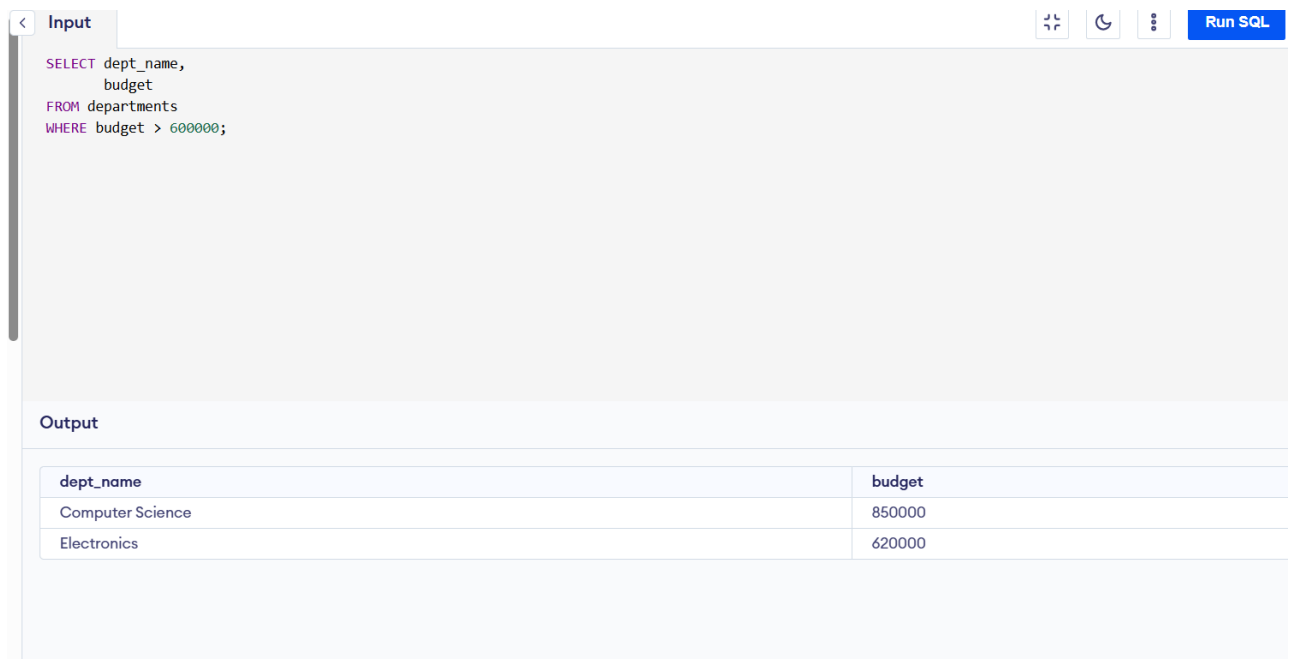
dept_name	average_salary
Computer Science	91500
Electronics	82000
Mechanical	79000
Civil	76000

32. Departments where total budget exceeds 600000

SQL

```
SELECT dept_name,
       budget
FROM departments
WHERE budget > 600000;
```

EXAMPLE TABLE



The screenshot shows a SQL query editor interface. At the top, there is a tab labeled 'Input' and a 'Run SQL' button. The query text is as follows:

```
SELECT dept_name,  
       budget  
FROM departments  
WHERE budget > 600000;
```

Below the query editor, there is an 'Output' section displaying a table with two columns: 'dept_name' and 'budget'.

dept_name	budget
Computer Science	850000
Electronics	620000

33. Grade distribution for course CS101

SQL

```
SELECT e.grade,  
       COUNT(*) AS grade_count  
FROM enrollments e  
JOIN courses c  
ON e.course_id = c.course_id  
WHERE c.course_code = 'CS101'  
GROUP BY e.grade;
```

EXAMPLE TABLE



The screenshot shows a SQL IDE interface. The 'Input' tab at the top contains the following SQL query:

```
SELECT e.grade,  
       COUNT(*) AS grade_count  
FROM enrollments e  
JOIN courses c  
ON e.course_id = c.course_id  
WHERE c.course_code = 'CS101'  
GROUP BY e.grade;
```

Below the query editor, the 'Output' tab displays the results in a table:

grade	grade_count
A	2
B	2

34. Departments with more than 2 students enrolled

SQL

```
SELECT d.dept_name,  
       COUNT(s.student_id) AS total_students  
FROM departments d  
JOIN students s  
ON d.department_id = s.department_id  
GROUP BY d.department_id, d.dept_name  
HAVING COUNT(s.student_id) > 2;
```

EXAMPLE TABLE

Input

SQL Editor

```
SELECT d.dept_name,
       COUNT(s.student_id) AS total_students
FROM departments d
JOIN students s
ON d.department_id = s.department_id
GROUP BY d.department_id, d.dept_name
HAVING COUNT(s.student_id) > 2;
```

Run SQL

Output

dept_name	total_students
Computer Science	5
Electronics	3